

Competitive Programming

Section 17 Editorial: Topological Sorting And DAG DP

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	4
2	Kahn's algorithm reminder	5
3	A. Lexicographic Course Order	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Walk-through	6
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	8
3.8	Full C++ solution	8
3.9	Full Python solution	9
3.10	Common wrong answers	10
3.11	Tests that matter	10
4	B. Longest DAG Path	11
4.1	Restatement	11
4.2	Constraints and traps	11
4.3	Observations	11
4.4	Walk-through	11
4.5	Algorithm	12
4.6	Correctness proof	12
4.7	Complexity	12
4.8	Full C++ solution	13
4.9	Full Python solution	14
4.10	Common wrong answers	14
4.11	Tests that matter	14
5	C. Project Schedule	16
5.1	Restatement	16
5.2	Constraints and traps	16
5.3	Observations	16
5.4	Walk-through	16
5.5	Algorithm	17
5.6	Correctness proof	17
5.7	Complexity	18
5.8	Full C++ solution	18
5.9	Full Python solution	19
5.10	Common wrong answers	20
5.11	Tests that matter	20
6	Stress testing	21

1 Editorial overview

All three problems are dependency-graph problems:

Problem	Main tool
A. Lexicographic Course Order	Kahn's algorithm with min-heap
B. Longest DAG Path	Kahn's algorithm plus DP
C. Project Schedule	Kahn's algorithm plus earliest-time DP

The repeated pattern is:

```
indegree counts unmet prerequisites
indegree 0 means currently available
removing a vertex reduces indegree of its outgoing neighbors
```

2 Kahn's algorithm reminder

For a directed graph:

```
indeg[v] = number of incoming edges to v
```

Algorithm:

```
put all indeg=0 vertices into a queue or heap
while available set is not empty:
    remove one available vertex u
    append/process u
    for each edge u -> v:
        indeg[v] -= 1
        if indeg[v] == 0:
            add v to available set
```

If fewer than n vertices are processed, the graph has a directed cycle.

3 A. Lexicographic Course Order

3.1 Restatement

Find a valid order of all courses so every prerequisite comes first. If several orders exist, output the lexicographically smallest one. If a cycle makes that impossible, print **IMPOSSIBLE**.

3.2 Constraints and traps

The graph is directed. A rule **a b** means:

```
a -> b
```

Traps:

- using a normal queue gives some topological order, not necessarily the lexicographically smallest;
- cycle detection is required;
- courses are 1-indexed in input but arrays are 0-indexed in code.

3.3 Observations

At any step, the next course must have no remaining prerequisites. Among all currently available courses, choosing the smallest course number gives the lexicographically smallest order.

This is exactly Kahn's algorithm with a min-heap.

3.4 Walk-through

Dependencies:

```
1 -> 3
2 -> 3
2 -> 4
4 -> 5
```

Available courses start as:

```
1,2
```

The smallest is 1, then 2. After 2, courses 3 and 4 both become available, so choose 3 first. The final order is:

```
1 2 3 4 5
```

3.5 Algorithm

```
build adjacency list and indegree array
heap = all vertices with indegree 0
order = []

while heap not empty:
    u = pop smallest
    append u
    for v in adj[u]:
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if len(order) != n:
    print IMPOSSIBLE
else:
    print order
```

3.6 Correctness proof

Lemma 1. Every course popped from the heap has all prerequisites already in the output order.

Proof. A course enters the heap only when its indegree becomes 0. The indegree counts prerequisites not yet processed. Therefore no unprocessed prerequisite remains when the course is popped. $\square$

Lemma 2. If a valid course remains available at some step, it is in the heap.

Proof. A course is available exactly when all incoming prerequisite edges come from already processed courses. Kahn's algorithm decrements indegree for each processed predecessor, so its indegree is then 0, and the course is pushed into the heap. $\square$

Lemma 3. The algorithm's order is lexicographically smallest among all valid orders.

Proof. At the first position where two valid orders could differ, the prefix already chosen by the algorithm has been processed. The next valid course must be among the currently available courses. By Lemma 2, all such courses are in the heap, and the algorithm picks the smallest. No valid order can choose a smaller next course. $\square$

Theorem. The algorithm prints the lexicographically smallest valid order, or **IMPOSSIBLE** if none exists.

Proof. If all vertices are processed, Lemma 1 proves the order is valid, and Lemma 3 proves it is lexicographically smallest. If not all vertices are processed, the remaining vertices never reach

indegree 0, which can only happen because of a directed cycle among remaining prerequisites. No valid topological order exists. \square

3.7 Complexity

Time: $O((n + m) \log n)$
Memory: $O(n + m)$

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        --a;
        --b;
        adj[a].push_back(b);
        indeg[b]++;
    }

    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) pq.push(i);
    }

    vector<int> order;
    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) pq.push(v);
        }
    }
}
```

```

if ((int)order.size() != n) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int i = 0; i < n; i++) {
        if (i) cout << ' ';
        cout << order[i] + 1;
    }
    cout << '\n';
}
return 0;
}

```

3.9 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1

    heap = [i for i in range(n) if indeg[i] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        u = heapq.heappop(heap)
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                heapq.heappush(heap, v)

    if len(order) != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(str(x + 1) for x in order))

```

```
if __name__ == "__main__":  
    main()
```

3.10 Common wrong answers

- Using a FIFO queue when lexicographic smallest order is required.
- Incrementing indegree of **a** instead of **b**.
- Forgetting to print IMPOSSIBLE when not all vertices are processed.
- Outputting zero-based course numbers.

3.11 Tests that matter

Use:

- no dependencies;
- one unique chain;
- multiple available courses at once;
- directed cycle;
- disconnected dependency graph.

4 B. Longest DAG Path

4.1 Restatement

Given a directed acyclic graph, find the largest number of edges on any directed path.

4.2 Constraints and traps

The graph is guaranteed acyclic. That guarantee is essential.

Traps:

- BFS shortest-path thinking is the wrong direction; this asks for longest path;
- longest path in a general directed graph is not solved by this DP;
- vertices with no incoming edges should start with $dp = 0$.

4.3 Observations

Define:

```
dp[v] = longest path length ending at v
```

For each edge $u \rightarrow v$, any longest path ending at u can be extended to v :

```
dp[v] = max(dp[v], dp[u] + 1)
```

Topological order guarantees that when we process u , all information needed for $dp[u]$ is already final.

4.4 Walk-through

Graph:

```
1 -> 2 -> 4 -> 5
|      |
v      v
3      5
```

Updates:

```
1->2: dp[2] = 1
1->3: dp[3] = 1
2->4: dp[4] = 2
2->5: dp[5] = 2
4->5: dp[5] = 3
```

Answer: 3.

4.5 Algorithm

```
build graph and indegrees
queue = all indegree-0 vertices
dp = zeros

while queue not empty:
    u = pop
    for v in adj[u]:
        dp[v] = max(dp[v], dp[u] + 1)
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

print max(dp)
```

4.6 Correctness proof

Lemma 1. When a vertex u is popped by Kahn's algorithm in a DAG, every predecessor of u has already been processed.

Proof. u is popped only after its indegree becomes 0. Each incoming edge is removed only when its predecessor is processed. $\square$

Lemma 2. When vertex u is popped, $dp[u]$ equals the longest path length ending at u .

Proof. Every path ending at u either has length 0 if it contains only u , or ends with some edge $p \rightarrow u$. By Lemma 1, every predecessor p has already been processed, and the transition $dp[u] = \max(dp[u], dp[p] + 1)$ was applied. Therefore all possible last edges into u have been considered. $\square$

Theorem. The algorithm prints the length of the longest directed path in the DAG.

Proof. By Lemma 2, every $dp[v]$ is the longest path ending at v . Every directed path ends at some vertex, so the maximum over all vertices is the global longest path length. $\square$

4.7 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
        indeg[v]++;
    }

    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) q.push(i);
    }
    vector<int> dp(n, 0);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            dp[v] = max(dp[v], dp[u] + 1);
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    cout << *max_element(dp.begin(), dp.end()) << '\n';
    return 0;
}
```

4.9 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        indeg[v] += 1

    q = deque(i for i in range(n) if indeg[i] == 0)
    dp = [0] * n
    while q:
        u = q.popleft()
        for v in adj[u]:
            dp[v] = max(dp[v], dp[u] + 1)
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    print(max(dp, default=0))

if __name__ == "__main__":
    main()
```

4.10 Common wrong answers

- Starting `dp` at 1 and accidentally counting vertices instead of edges.
- Processing vertices in numeric order instead of topological order.
- Trying to use shortest-path BFS.
- Forgetting isolated vertices, whose longest ending path is 0.

4.11 Tests that matter

Use:

- no edges;

- one chain;
- diamond graph with converging paths;
- disconnected DAG;
- edges from high-numbered to low-numbered vertices.

5 C. Project Schedule

5.1 Restatement

Each job has a duration. If $a \rightarrow b$, job b can start only after job a finishes. Compute earliest finish time for every job, or print IMPOSSIBLE if there is a dependency cycle.

5.2 Constraints and traps

Durations can be large and paths can contain many jobs, so C++ needs `long long`.

Traps:

- use maximum predecessor finish time, not sum of predecessor finish times;
- cycle detection is required;
- jobs with no prerequisites start at day 0;
- output finish times in original job order, not topological order.

5.3 Observations

For every job:

```
earliest_start[v] = max(finish[u] for every prerequisite u of v)
finish[v] = earliest_start[v] + duration[v]
```

Kahn's algorithm processes each prerequisite before its dependents, so we can push finish times forward along edges.

5.4 Walk-through

Durations:

```
job:      1  2  3  4  5
duration: 3  2  4  6  1
```

Dependencies:

```
1 -> 3
2 -> 3
3 -> 4
2 -> 5
5 -> 4
```

Jobs 1 and 2 start at day 0:

```
finish[1] = 3
finish[2] = 2
```

Job 3 waits for jobs 1 and 2, so it starts at $\max(3, 2) = 3$ and finishes at 7.

Job 5 waits for job 2, so it finishes at 3.

Job 4 waits for jobs 3 and 5, so it starts at $\max(7, 3) = 7$ and finishes at 13.

5.5 Algorithm

```
build graph and indegrees
earliest_start = zeros
finish = duration copy
queue = all indegree-0 jobs
processed = 0

while queue not empty:
    u = pop
    processed += 1
    finish[u] = earliest_start[u] + duration[u]
    for v in adj[u]:
        earliest_start[v] = max(earliest_start[v], finish[u])
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if processed != n:
    print IMPOSSIBLE
else:
    print finish times
```

5.6 Correctness proof

Lemma 1. When job u is processed, $\text{earliest_start}[u]$ equals the maximum finish time among all prerequisites of u .

Proof. Kahn's algorithm processes a job only after all incoming prerequisite edges have been removed, meaning every prerequisite has already been processed. Each prerequisite p applies $\text{earliest_start}[u] = \max(\text{earliest_start}[u], \text{finish}[p])$. Thus all prerequisite finish times have been considered, and no non-prerequisite can increase the value. $\square$

Lemma 2. When job u is processed, $\text{finish}[u]$ is the earliest possible finish time of u .

Proof. Job u cannot start before every prerequisite finishes, so it cannot start before the maximum prerequisite finish time. By Lemma 1, that is $\text{earliest_start}[u]$. Starting at that time is feasible because all prerequisites are done, so finishing at $\text{earliest_start}[u] + \text{duration}[u]$ is earliest possible. $\square$

Theorem. The algorithm prints correct earliest finish times, or IMPOSSIBLE if scheduling all jobs is impossible.

Proof. If all jobs are processed, Lemma 2 proves every printed finish time is correct. If not all jobs are processed, the remaining jobs are blocked by a directed cycle of prerequisites, so no schedule can complete all jobs. $\square$

5.7 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

5.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> duration(n);
    for (long long &x : duration) cin >> x;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        --a;
        --b;
        adj[a].push_back(b);
        indeg[b]++;
    }

    vector<long long> earliest_start(n, 0), finish = duration;
    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) q.push(i);
    }

    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        processed++;
    }
```

```

        finish[u] = earliest_start[u] + duration[u];
        for (int v : adj[u]) {
            earliest_start[v] = max(earliest_start[v], finish[u]);
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }

    if (processed != n) {
        cout << "IMPOSSIBLE\n";
    } else {
        for (int i = 0; i < n; i++) {
            if (i) cout << ' ';
            cout << finish[i];
        }
        cout << '\n';
    }
    return 0;
}

```

5.9 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    duration = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1

    earliest_start = [0] * n
    finish = duration[:]
    q = deque(i for i in range(n) if indeg[i] == 0)
    processed = 0
    while q:
        u = q.popleft()

```

```

    processed += 1
    finish[u] = earliest_start[u] + duration[u]
    for v in adj[u]:
        earliest_start[v] = max(earliest_start[v], finish[u])
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)

    if processed != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(map(str, finish)))

if __name__ == "__main__":
    main()

```

5.10 Common wrong answers

- Summing all prerequisite finish times instead of taking the maximum.
- Forgetting that independent jobs can run in parallel from day 0.
- Printing in topological order instead of job-number order.
- Missing cycle detection.
- Overflowing `int` in C++.

5.11 Tests that matter

Use:

- no dependencies;
- one long chain;
- multiple prerequisites into one job;
- cycle;
- large durations.

6 Stress testing

Problem A:

```
generate small directed graphs  
compare heap Kahn output to brute-force enumeration of all permutations
```

Problem B:

```
generate DAGs using a hidden random order  
compare topological DP to brute-force DFS over all paths on small graphs
```

Problem C:

```
generate DAGs and cyclic graphs  
for DAGs, compare against repeated relaxation in topological order  
for cyclic graphs, require IMPOSSIBLE
```

The local generators cover acyclic and cyclic cases where relevant, and the checker runs both C++ and Python reference implementations.

7 What to carry forward

Topological sorting is the bridge from graph traversal to graph DP.

Carry these rules forward:

```
indegree means unmet prerequisites  
topological order means all dependencies point forward  
DAG DP is just normal DP in a dependency-respecting order  
cycle detection is order-size < n in Kahn's algorithm
```

The next graph sections build on this dependency thinking with DSU, minimum spanning trees, and weighted shortest paths.